

=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===

=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===

[Файл] wiseIPList.cs

=== СОДЕРЖИМОЕ ФАЙЛОВ ===

---wiseIPList.cs---

```
using System;
using System.Collections.Generic;
using System.ComponentModel.Design;
using System.IO;
using System.Linq;
using System.Net;
using System.Net.Security;
using System.Runtime.CompilerServices;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using Windows.UI.Input.Inking.Preview;
using static ipinpool.wiseIPList;

namespace ipinpool
{
    public class wiseIPList
    {

        public List<IPclass> ipTable { get; set; } = new List<IPclass>();
        public List<IPclass> PoolTable { get; set; } = new List<IPclass>();
        public List<IPclass> Filter { get; set; } = new List<IPclass>(); //Адреса исключаемые при разборе
        логa
        public void ParseFile(string filename)
        {
            if (!File.Exists(filename)) return;
            //FileStream rdr = TextReader(filename);
            FileStream frdr = File.Open(filename, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
            TextReader reader = new StreamReader(frdr);
            string? line = reader.ReadLine();

            while (line != null)
            {
                if (IPclass.TryParse(line, out IPclass ip))
                {
                    if (ip.IsPool) { PoolTable.Add(ip); }
                    else { ipTable.Add(ip); }

                }
                line = reader.ReadLine();
            }
            reader.Close();
            frdr.Close();
        }
        public void Sort(ref List<IPclass> list)
        {
            for (int i = 0; i < list.Count-1; i++)
                for(int j=i+1; j<list.Count; j++)
                {
                    if (list[i].IsAbove(list[j]))
                    {
                        IPclass ip = list[i];
                        list[i] = list[j];
                        list[j] = ip;
                    }
                }
        }
        public void AddPool(IPclass? ip)
        {
            if(ip!=null)
            {
                if(ip.IsPool)
                {
                    PoolTable.Add(ip);
                }
            }
        }
    }
}
```

```
}  
}  
}
```

```
public void AddAddress(IPclass? ip)  
{  
    List<IPclass> iptable = this.ipTable;  
    AddAddressTo(ip, ref iptable);  
}  
public void AddAddressTo(IPclass? ip, ref List<IPclass> iptable)  
{  
    if(ip==null) return;  
    // if (ipTable.IndexOf(ip) > 0) return;  
    // if(Filter.IndexOf(ip) > 0) return;  
    if (AddrInTable(ip, iptable)) return;  
    if (AddrInTable(ip, Filter)) return;  
    iptable.Add(ip);  
    // 3.Вызываем событие, если на него кто-то подписался  
    OnAddressAdded?.Invoke(ip);  
}  
private bool AddrInTable(IPclass ip, List<IPclass> Table)  
{  
    foreach(IPclass p in Table)  
    {  
        bool r = p.IPinPool(ip);  
        if (r) return true;  
    }  
    return false;  
}  
public void ParseEDLogs(string dir, bool newlist = true)  
{  
    //F:\SteamLibrary\steamapps\common\Elite Dangerous\Products\elite-dangerous-odyssey-64\Logs  
    string local_dir = "F:\\SteamLibrary\\steamapps\\common\\Elite Dangerous\\Products\\elite-  
dangerous-odyssey-64\\Logs";  
    if(dir!=string.Empty) local_dir = dir.ToLower();  
    int index = 0;  
    if(Directory.Exists(local_dir))  
    {  
        if(newlist) ipTable.Clear();  
        string[] files = Directory.GetFiles(local_dir, "*.log");  
        OnParseStart?.Invoke(this, new WIP_Parse_StartEventArgs(files.Length));  
        foreach (string file in files)  
        {  
  
            OnParseProceed?.Invoke(this, new WIP_Parse_ProceedEventArgs(index, file));  
            parseFile(file);  
            index++;  
        }  
    }  
    OnParseComplete?.Invoke(this, new EventArgs());  
}  
public List<IPclass> ParseString(string text)  
{  
    List<IPclass> rez = new List<IPclass>();  
    string[] strings = text.Split("\n");  
    foreach (string s in strings)  
    {  
        string ip_str = ipSearch(s);  
        if (ip_str != null)  
        {  
            AddAddressTo(IPclass.Parse(ip_str), ref rez);  
        }  
    }  
    return rez;  
}  
public void ParseEDLogsAsync(string dir, bool newlist = true)  
{  
    //await ParseEDLogs(dir, newlist);  
    Task task = new Task(()=>ParseEDLogs(dir, newlist));  
    task.Start();  
}  
  
private void parseFile(string filename)
```

```

{
try
{
FileStream fs = File.Open(filename, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
TextReader reader = new StreamReader(fs);
string? currentLine = String.Empty;
currentLine = reader.ReadLine();

while (currentLine != null)
{
// 1. АВТОМАТИЧЕСКИЙ ДИНАМИЧЕСКИЙ ФИЛЬТР (Убираем рутину пользователя)
// Ищем маркеры локального внешнего адреса или STUN-точки выхода
if (currentLine.Contains("WAN:") ||
currentLine.Contains("STUN mapped address is"))

{
// Выдергиваем IP-адрес из этой конкретной технической строки
string filterIpStr = ipSearch(currentLine);
if (!string.IsNullOrEmpty(filterIpStr))
{
IPclass? filterIpObj = IPclass.Parse($"{filterIpStr}/32");
if (filterIpObj != null)
{
// Если этого адреса еще нет в черном списке — добавляем его в Filter!
// Метод AddrInTable у тебя уже написан на странице 2.
if (!AddrInTable(filterIpObj, this.Filter))
{
this.Filter.Add(filterIpObj);
// Можно отправить лог в отладку, чтобы видеть, что автофильтр сработал
System.Diagnostics.Debug.WriteLine($"[AutoFilter] Локальный/STUN адрес изолирован: {filterIpStr}");
}
}
}
}

// 2. БОЕВОЙ ПАРСИНГ ИГРОВЫХ ХОСТОВ (Остается штатным)
string ip_str = ipSearch(currentLine);
if (!string.IsNullOrEmpty(ip_str))
{
// Твой метод AddAddress сам сличит адрес с обновленным списком Filter
// и заблокирует добавление твоего WAN/STUN IP в общую таблицу!
AddAddress(IPclass.Parse(ip_str));
}

currentLine = reader.ReadLine();
}
reader.Close();
fs.Close();
}
catch (Exception e)
{
Console.WriteLine(e.ToString());
}
}

private string ipSearch(string logEntry)
{
Regex reg = new Regex("(2[0-4]\\d|25[0-5]|[01]?\\d\\d?)\\.\\.\\.\\{3\\}(2[0-4]\\d|25[0-5]|[01]?\\d\\d?)");
if (reg.IsMatch(logEntry))
{
return reg.Match(logEntry).ToString();
}
return String.Empty;
}

public void SaveTo(string filename, bool mikrotik_prep =true)
{
try
{
StreamWriter wr = new StreamWriter(filename);

foreach (IPclass ip in ipTable)

```

```

{
string ws = string.Empty;
if(mikrotik_prep)
{
ws = "/ip firewall address-list add address=" + ip.ToString() + " list=ED";
}
else
{
ws = ip.ToString();
}
wr.WriteLine(ws);
}
wr.Flush();
wr.Close();
}
catch (Exception e) { Console.WriteLine(e.Message); }

}
public bool LoadFilters(string filename)
{
//список фильтров представляет собой текстовый файл вида IP/Mask
if (File.Exists(filename))
{
try
{
this.Filter.Clear();
TextReader reader = new StreamReader(filename);
string? line = reader.ReadLine();
while (line != null)
{
if(line.IndexOf("#")>-1)
{
line = line.Substring(0, line.IndexOf("#"));
}
IPclass? ip = IPclass.Parse(line);
if (ip != null)
{
if(!AddrInTable(ip, this.Filter)) this.Filter.Add(ip);
}
line = reader.ReadLine();
}
reader.Close();
return true;
}
catch
{ return false; }

}
else return false;
}

public void SetDefaultFilters(string filename)
{
try
{
TextWriter writer = new StreamWriter(filename);
writer.WriteLine("#cut off 0.0.0.0/32 ");
writer.WriteLine("0.0.0.0/32");
writer.WriteLine("#cut off local network areas");
writer.WriteLine("10.0.0.0/8");
writer.WriteLine("172.16.0.0/12");
writer.WriteLine("192.168.0.0/16");
writer.WriteLine("#cut off routers");

writer.WriteLine("#place addresses to exclude below IP/Mask");
writer.Flush();
writer.Close();
}
catch { }
}

public delegate void LogParseStart(object sender, WIP_Parse_StartEventArgs e);
public delegate void LogParseProceed(object sender, WIP_Parse_ProceedEventArgs e);
public delegate void LogParseComplete(object sender, EventArgs e);

```

```

public delegate void AddressAddedEventHandler(IPclass newAddress);

public event LogParseStart? OnParseStart;
public event LogParseProceed? OnParseProceed;
public event LogParseComplete? OnParseComplete;
public event AddressAddedEventHandler? OnAddressAdded;
}
public class WIP_Parse_StartEventArgs:EventArgs
{
public int FilesCount { get; private set; } = 0;

public WIP_Parse_StartEventArgs(int filesCount)
{
this.FilesCount = filesCount;
}
}
public class WIP_Parse_ProceedEventArgs : EventArgs
{
public int FileIndex { get; private set; } = 0;
public string Filename { get; private set; } = string.Empty;
public WIP_Parse_ProceedEventArgs( int fileIndex, string filename)
{
FileIndex = fileIndex;
Filename = filename;
}
}

public class IPclass
{
// Заменяем автосвойство на жесткое приватное поле.
// Это на 100% заставит компилятор в Release выделять новый изолированный массив для каждого
объекта!
private byte[] _ipa = new byte[4] { 0, 0, 0, 0 };

// Публичное свойство теперь просто ссылается на наше изолированное поле
public byte[] IPA
{
get { return _ipa; }
set { _ipa = value; }
}

// Чистый, стандартный конструктор
public IPclass()
{
// Принудительно выделяем персональный массив из 4 байт при рождении каждого объекта
_ipa = new byte[4] { 0, 0, 0, 0 };
}

public int Port { get; set; } = 0;
public int PoolSize { get; set; } = 32;
public bool IsPool { get { return PoolSize < 32; } }

private int _inPoolCounter = 0;
public int InPoolCounter { get { return _inPoolCounter; } }
// Измени заголовок и концовку метода TryParse на обычный возврат объекта
public static IPclass? Parse(string str)
{
if (string.IsNullOrEmpty(str)) return null;
if (str.Trim().ToLower().IndexOf("#") > -1) return null;

string tmps = str.ToLower().Replace("add address=", "");
int ioflist = tmps.IndexOf("list");
if (ioflist > -1)
{
tmps = tmps.Substring(0, ioflist);
}
tmps = tmps.Trim();
string ipstr = tmps;

// Создаем строго изолированный объект внутри этого вызова
IPclass pc = new IPclass();

```

```

if (tmps.IndexOf("/") > 0)
{
    string[] ipstrs = tmps.Split("/");
    ipstr = ipstrs[0];
    if (int.TryParse(ipstrs[1], out int ps))
    {
        pc.PoolSize = ps;
    }
}

if (tmps.IndexOf(":") > 0)
{
    string[] ipstrs = tmps.Split(":");
    ipstr = ipstrs[0];
    if (int.TryParse(ipstrs[1], out int ps))
    {
        pc.Port = ps;
    }
}

string[] v = ipstr.Split(".");
if (v.Length != 4) return null;

byte[] tempIpa = new byte[4];
int id = 0;
foreach (string s in v)
{
    if (byte.TryParse(s, out byte b))
    {
        tempIpa[id] = b;
        id++;
    }
    else
    {
        return null;
    }
}

// Принудительно выделяем новый массив для конкретного объекта
pc.IPA = new byte[] { tempIpa[0], tempIpa[1], tempIpa[2], tempIpa[3] };
return pc;
}

// Старый метод TryParse можно просто перенаправить на новый Parse для совместимости:
public static bool TryParse(string str, out IPclass pc)
{
    var res = Parse(str);
    pc = res ?? new IPclass();
    return res != null;
}

public override string ToString()
{
    string rez = string.Empty;
    foreach (byte b in IPA)
    {
        if (rez.Length > 0) { rez += "." + b.ToString(); } else { rez = b.ToString(); }
    }
    if(PoolSize>1)
    {
        rez += "/" + PoolSize.ToString();
    }
    if(Port>0)
    {
        rez += ":" + Port.ToString();
    }
    return rez;
}

public bool LongIPMask { get; set; } = false;
public bool IPinPool(IPclass ip)

```

```

{
if (this.PoolSize < 8)
{
byte hr = getHighRange(this.IPA[0], this.PoolSize);
if ((ip.IPA[0] >= this.IPA[0]) && (ip.IPA[0] <= hr))
{
_inPoolCounter++;
return true;
}
return false;
}
if ((this.PoolSize >= 8) && (this.PoolSize < 16))
{
byte hr = getHighRange(this.IPA[1], this.PoolSize-8);
if (ip.IPA[0] == this.IPA[0])
if ((ip.IPA[1] >= this.IPA[1]) && (ip.IPA[1] <= hr))
{
_inPoolCounter++;
return true;
}
return false;
}
if ((this.PoolSize >= 16) && (this.PoolSize < 24))
{
byte hr = getHighRange(this.IPA[2], this.PoolSize - 16);
if ((ip.IPA[0] == this.IPA[0]) && (ip.IPA[1] == this.IPA[1]))
if ((ip.IPA[2] >= this.IPA[2]) && (ip.IPA[2] <= hr))
{
_inPoolCounter++;
return true;
}
return false;
}
if ((this.PoolSize >= 24) && (this.PoolSize < 32))
{
byte hr = getHighRange(this.IPA[3], this.PoolSize - 24);
if ((ip.IPA[0] == this.IPA[0]) && (ip.IPA[1] == this.IPA[1]) && (ip.IPA[2] == this.IPA[2]))
if ((ip.IPA[3] >= this.IPA[3]) && (ip.IPA[3] <= hr))
{
_inPoolCounter++;
return true;
}
return false;
}
if (this.PoolSize >= 32)
{
for (int i = 0; i < this.IPA.Length; i++)
{
if (this.IPA[i] != ip.IPA[i]) return false;
}
_inPoolCounter++;
return true;
}

return false;

}

private byte getHighRange(byte low, int mask)
{
byte rez = 0; // 0 1 2 3 4 5 6 7
byte[] maskTable = { 255, 127, 63, 31, 15, 7, 3, 1 };
rez = (byte)(low+maskTable[mask]);
return rez;
}

public bool IsAbove(IPclass ip)
{
for (int i = 0; i < this.IPA.Length; i++)
{
if (ip.IPA[i] < this.IPA[i])
{
return true;
}
if (ip.IPA[i] > this.IPA[i])
{

```

```

return false;
}
}
return false;
}
public string IP { get {
string rez = string.Empty;
foreach (byte b in this.IPA)
{
if(rez==string.Empty)
{
rez = b.ToString();
}
else
{
rez += "." + b.ToString();
}
}
return rez;
} }
public string Mask { get {
if (!LongIPMask)
{ return this.PoolSize.ToString(); }
else return IPMask.GetStrIPMask(this.PoolSize);
} }
public string Size
{
get
{
if (PoolSize == 32) return "1";
if (PoolSize == 31) return "2";

// Для всех остальных подсетей (30, 29, 24 и т.д.) вычитаем 2 служебных адреса
long totalAddresses = (long)Math.Pow(2, 32 - PoolSize);
long usableAddresses = totalAddresses - 2;

return usableAddresses.ToString();
}
}

}

public class IPMask
{
static byte[] masks = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32 };

static public string GetStrIPMask(int mask)
{
string rez = string.Empty;
if (mask < 0 || mask > 31) return "255.255.255.255";
if(mask < 32&&mask >=24)
{
rez = "255.255.255.";
int m = mask-24;
rez += ByteMaskToValue(m);
}
if(mask<24&&mask>=16)
{
rez = "255.255.";
int m = mask - 16;
rez += ByteMaskToValue(m);
rez += ".0";
}
if(mask<16&&mask>=8)
{
rez = "255.";
int m = mask - 8;
rez += ByteMaskToValue(m);
rez += ".0.0";
}
return rez;
}
static string ByteMaskToValue(int mask)

```



```
{  
int b = 0;  
b = 2 ^ mask;  
return b.ToString();  
}  
}  
}
```